

AZ-400: Designing and Implementing Microsoft DevOps Solutions

Complete Exam Study Guide

Exam Details:

- **Exam Code:** AZ-400
 - **Certification:** Microsoft Certified: DevOps Engineer Expert
 - **Prerequisite:** Must have either AZ-104 (Azure Administrator) or AZ-204 (Azure Developer) certification
 - **Passing Score:** 700/1000
 - **Question Types:** Multiple choice, case studies, drag-and-drop, hot area, build list
-

Exam Domain Weightages

Domain	Weight
Configure processes and communications	10-15%
Design and implement source control	15-20%
Design and implement build and release pipelines	40-45%
Develop a security and compliance plan	10-15%
Implement an instrumentation strategy	10-15%

Domain 1: Configure Processes and Communications (10-15%)

1.1 Configure Activity Traceability and Flow of Work

Azure Boards Overview

- **Work item types:** Epic → Feature → User Story/Product Backlog Item → Task/Bug
- **Process templates:**
 - **Basic:** Simplest, Issue → Task
 - **Agile:** User Story, Bug, Task, Feature, Epic
 - **Scrum:** Product Backlog Item, Bug, Task, Feature, Epic
 - **CMMI:** Requirement, Bug, Task, Feature, Epic (formal change management)

Configuring Work Item Tracking

- **Area paths:** Organize work by team, product, or feature area
- **Iteration paths:** Define sprints/iterations for time-boxed work
- **Custom fields:** Add fields via Organization Settings → Process → Work Item Types
- **Custom rules:** Automate field behaviors (e.g., set field when state changes)
- **Custom states:** Add workflow states beyond default

Queries and Dashboards

- **Query types:**
 - Flat list: Simple list of work items
 - Work items and direct links: Parent-child relationships
 - Tree of work items: Full hierarchy
- **Query operators:** (=, <>, >, <, Contains, In, Was Ever, Changed, Under) (for area/iteration paths)
- **Dashboard widgets:** Burndown, Velocity, Cumulative Flow, Query Results, Chart for Work Items

Traceability

- **Link types:**
 - Parent/Child: Hierarchical relationships
 - Related: General association
 - Predecessor/Successor: Dependencies
 - Tested By/Tests: Links to test cases
 - Build/Integrated in Build: CI links
 - Pull Request: Code review links
- **End-to-end traceability:** Requirement → Code → Build → Test → Release

GitHub Integration

- **AB#[ID]:** Link commits/PRs to Azure Boards work items
- **Azure Boards app for GitHub:** Install from GitHub Marketplace
- **Sync options:** One-way (GitHub → Azure Boards) or bidirectional
- **Keywords in commits:** Fixes AB#123, Closes AB#456

Microsoft Learn: <https://learn.microsoft.com/en-us/azure/devops/boards/>

1.2 Configure Collaboration and Communication

Azure DevOps Wikis

- **Project Wiki:** Markdown-based, version controlled
- **Publish code as wiki:** Use repository folder as wiki source
- **Wiki features:**
 - Table of contents auto-generation
 - Mermaid diagram support
 - Math formulas (LaTeX)
 - Work item mentions: `#123`
 - User mentions: `@username`

Team Notifications

- **Subscription types:**
 - Personal: Individual user settings
 - Team: Team-level notifications
 - Project: Project-wide notifications
 - Organization: Global notifications
- **Delivery options:** Email, SOAP, REST API, Service Hooks
- **Custom subscriptions:** Filter by area path, work item type, state changes

Service Hooks

- **Supported services:** Slack, Microsoft Teams, Jenkins, Trello, Grafana, Datadog, etc.
- **Event types:** Work item created/updated, code pushed, build completed, release deployed
- **Configuration:** Project Settings → Service Hooks
- **Webhooks:** Custom HTTP POST to any endpoint

Microsoft Teams Integration

- **Azure Pipelines app:** Build/release notifications in channels
- **Azure Boards app:** Work item cards, create work items from Teams
- **Azure Repos app:** Code notifications, PR discussions
- **Commands:** `@azure boards create`, `@azure pipelines run`

Slack Integration

- **Azure DevOps app for Slack:** `/azboards`, `/azpipelines`, `/azrepos`
- **Features:** Subscribe to notifications, create work items, trigger pipelines

Microsoft Learn: <https://learn.microsoft.com/en-us/azure/devops/project/wiki/>

1.3 Integrate Azure DevOps and GitHub with Microsoft Tools

Azure DevOps and Microsoft Entra ID

- **Tenant connection:** Organization Settings → Microsoft Entra ID
- **Benefits:** SSO, conditional access, identity governance
- **User management:** Sync users/groups from Entra ID
- **Access policies:** Set at organization level

Power BI Integration

- **OData feed:** Connect Power BI to Azure DevOps Analytics
- **Analytics views:** Pre-built or custom views for reporting
- **Dashboards:** Embed Power BI reports in Azure DevOps dashboards
- **Data available:** Work items, pipelines, test results, code metrics

Power Automate Integration

- **Azure DevOps connector:** Triggers and actions for automation
- **Common flows:**
 - Create work item when form submitted
 - Notify team when build fails
 - Sync work items with external systems
 - Automate approvals

Power Apps Integration

- **Custom apps:** Build apps that interact with Azure DevOps
- **Use cases:** Custom work item entry, approval workflows, status dashboards

Domain 2: Design and Implement Source Control (15-20%)

2.1 Design and Implement Branching Strategies

Git Branching Models

Trunk-Based Development

- **Main branch:** Single long-lived branch (main/master)
- **Feature branches:** Short-lived (< 1 day ideally)
- **Best for:** CI/CD, experienced teams, small changes
- **Characteristics:**
 - Frequent commits to main
 - Feature flags for incomplete features
 - Minimal merge conflicts

GitFlow

- **Main branches:**
 - `main`: Production-ready code
 - `develop`: Integration branch for features
- **Supporting branches:**
 - `feature/*`: New features (from develop)
 - `release/*`: Release preparation (from develop)
 - `hotfix/*`: Production fixes (from main)
- **Best for:** Scheduled releases, version support

GitHub Flow

- **Main branch:** Always deployable
- **Feature branches:** All work in feature branches
- **Process:** Branch → Commit → PR → Review → Merge → Deploy
- **Best for:** Continuous deployment

Release Flow (Microsoft)

- **Main branch:** Always releasable
- **Feature branches:** Short-lived, merged via PR
- **Release branches:** Created at release time, cherry-pick fixes
- **Best for:** Large teams, scheduled releases

Branch Policies (Azure Repos)

- **Require minimum reviewers:** 1, 2, or more approvers
- **Check for linked work items:** Enforce traceability
- **Check for comment resolution:** All comments must be resolved
- **Limit merge types:** Squash, rebase, merge commit
- **Build validation:** Require successful build before merge
- **Status checks:** External service validations
- **Automatically include reviewers:** Code owners, specific users/groups
- **Reset votes on push:** Require re-approval after new commits

```
yaml

# Azure DevOps YAML for branch policy build
trigger: none
pr:
  branches:
    include:
      - main
      - develop
```

Branch Protection (GitHub)

- **Require pull request reviews:** Dismiss stale reviews, require code owner review
- **Require status checks:** CI must pass
- **Require conversation resolution:** All discussions resolved
- **Require signed commits:** GPG/SSH signature verification
- **Require linear history:** No merge commits
- **Include administrators:** Apply rules to admins
- **Restrict pushes:** Limit who can push

Microsoft Learn: <https://learn.microsoft.com/en-us/azure/devops/repos/git/branch-policies>

2.2 Configure and Manage Repositories

Repository Structure Strategies

Monorepo

- **Definition:** Single repository for multiple projects/services
- **Advantages:**
 - Simplified dependency management
 - Atomic changes across projects
 - Unified versioning
 - Easier code sharing
- **Disadvantages:**
 - Larger repository size
 - Longer clone times
 - Complex CI/CD pipelines
- **Tools:** Nx, Lerna, Bazel, Rush

Multi-repo (Polyrepo)

- **Definition:** Separate repository per project/service
- **Advantages:**
 - Independent versioning
 - Focused access control
 - Faster operations per repo
 - Team autonomy
- **Disadvantages:**
 - Complex dependency management
 - Cross-repo changes difficult
 - Duplicate configurations

Git Configuration

.gitignore

gitignore

Build outputs

bin/

obj/

dist/

build/

IDE files

.vs/

.vscode/

.idea/

*.suo

*.user

Dependencies

node_modules/

packages/

Environment files

.env

.env.local

appsettings.Development.json

Logs

*.log

logs/

.gitattributes


```
gitattributes
```

```
# Auto detect text files and normalize line endings
```

```
* text=auto
```

```
# Force specific file types
```

```
*.cs text eol=crlf
```

```
*.sh text eol=lf
```

```
*.bat text eol=crlf
```

```
# Binary files
```

```
*.png binary
```

```
*.jpg binary
```

```
*.pdf binary
```

```
# LFS tracked files
```

```
*.psd filter=lfs diff=lfs merge=lfs -text
```

```
*.zip filter=lfs diff=lfs merge=lfs -text
```

Git Large File Storage (LFS)

- **Purpose:** Store large files outside main repository
- **Setup:**

```
bash
```

```
git lfs install
```

```
git lfs track "*.psd"
```

```
git lfs track "*.zip"
```

```
git add .gitattributes
```

- **Commands:**
 - `git lfs ls-files`: List tracked files
 - `git lfs fetch`: Download LFS objects
 - `git lfs prune`: Remove old LFS objects

Git Hooks

- **Client-side hooks:**

- `pre-commit`: Run before commit (linting, tests)
- `prepare-commit-msg`: Modify commit message
- `commit-msg`: Validate commit message format
- `pre-push`: Run before push (tests, security scans)

- **Server-side hooks:**

- `pre-receive`: Before accepting push
- `update`: Per-branch validation
- `post-receive`: After push accepted

Scalar (Large Repository Optimization)

- **Features:** Partial clone, sparse checkout, commit graph, filesystem monitor
- **Setup:**

```
bash
```

```
scalar clone <repo-url>  
scalar register
```

- **Benefits:** Faster operations on repositories > 100GB

TFVC to Git Migration

- **Tools:**

- `git-tfs`: Preserve history
- Azure DevOps import: Built-in migration
- `git-tf`: Microsoft tool (deprecated)

- **Considerations:**

- Large binary files → Git LFS
- TFVC workspaces → Git clones
- Shelvesets → Git stash or branches

Microsoft Learn: <https://learn.microsoft.com/en-us/azure/devops/repos/git/>

2.3 Implement Pull Request Workflow

Pull Request Best Practices

- **Small PRs:** Easier to review, faster feedback
- **Descriptive titles:** What the change does
- **Detailed descriptions:** Why, how, testing done
- **Link work items:** `AB#123` or `Fixes #123`
- **Draft PRs:** Work in progress, not ready for review

PR Templates

markdown

```
<!-- .azuredevops/pull_request_template.md or .github/pull_request_template.md -->
```

Description

```
<!-- What does this PR do? -->
```

Related Work Items

```
<!-- AB#123 or Fixes #123 -->
```

Type of Change

- ☐ Bug fix
- ☐ New feature
- ☐ Breaking change
- ☐ Documentation update

Testing

- ☐ Unit tests added/updated
- ☐ Integration tests pass
- ☐ Manual testing completed

Checklist

- ☐ Code follows style guidelines
- ☐ Self-review completed
- ☐ Documentation updated
- ☐ No new warnings

Code Review Process

- **Review types:**
 - Approved: Ready to merge
 - Approved with suggestions: Minor changes optional
 - Wait for author: Significant changes required
 - Rejected: Fundamental issues
- **Review comments:**
 - Line comments: Specific code feedback
 - File comments: General file feedback
 - Overall comments: PR-level feedback
- **Suggested changes:** Reviewers can propose code changes

Auto-Complete

- **Azure DevOps:** Set PR to auto-complete when policies pass
- **Options:**
 - Complete linked work items
 - Delete source branch
 - Squash merge
 - Customize merge commit message

CODEOWNERS

```
text

# .github/CODEOWNERS or .azuredevops/CODEOWNERS

# Default owners for everything
* @default-team

# Specific paths
/src/api/ @api-team
/src/frontend/ @frontend-team
/docs/ @docs-team

# Specific files
*.sql @database-team
Dockerfile @platform-team
```

Microsoft Learn: <https://learn.microsoft.com/en-us/azure/devops/repos/git/pull-requests>

2.4 Configure Permissions in Source Control

Azure Repos Permissions

- **Repository-level:**
 - Read: Clone, fetch, read
 - Contribute: Push, create branches
 - Create branch: Create new branches
 - Create tag: Create tags
 - Manage permissions: Change permissions
 - Force push: Overwrite history (use cautiously)
 - Bypass policies: Skip branch policies
 - Remove others' locks: Unlock files locked by others
- **Branch-level:**
 - Inherit from repository
 - Specific branch permissions
 - Branch folder permissions (e.g., `feature/*`)

GitHub Permissions

- **Repository roles:**
 - Read: Clone, view, create issues
 - Triage: Manage issues/PRs without code access
 - Write: Push, manage branches
 - Maintain: Manage without admin access
 - Admin: Full access including settings
- **Organization roles:**
 - Member: Base permissions
 - Owner: Full administrative access

Sensitive Data in Repositories

- **Prevention:**

- Use `.gitignore` for sensitive files
- Use environment variables
- Use Azure Key Vault / GitHub Secrets
- Pre-commit hooks for secret scanning

- **Tools:**

- GitHub Secret Scanning: Automatic detection
- Azure DevOps Credential Scanner: Extension
- git-secrets: Pre-commit hook
- truffleHog: History scanning

- **Remediation:**

- Rotate exposed credentials immediately
- Use `git filter-branch` or BFG Repo-Cleaner to remove from history
- Consider repository as compromised

Microsoft Learn: <https://learn.microsoft.com/en-us/azure/devops/repos/git/set-git-repository-permissions>

Domain 3: Design and Implement Build and Release Pipelines (40-45%)

3.1 Design and Implement Pipeline Automation

Azure Pipelines YAML Structure

yaml

Basic pipeline structure

name: \$(Build.DefinitionName)_\$(Date:yyyyMMdd)\$(Rev:.r)

trigger:

branches:

include:

- main
- develop

exclude:

- feature/*

paths:

include:

- src/*

exclude:

- docs/*

tags:

include:

- v*

pr:

branches:

include:

- main

paths:

include:

- src/*

schedules:

- **cron:** "0 0 * * *"

displayName: Nightly build

branches:

include:

- main

always: true

pool:

vmImage: 'ubuntu-latest'

variables:

- **name:** buildConfiguration

value: 'Release'

- **group:** my-variable-group

- **template:** variables/common.yml

stages:

- stage: Build

displayName: 'Build Stage'

jobs:

- job: BuildJob

displayName: 'Build Application'

steps:

- task: DotNetCoreCLI@2

displayName: 'Restore packages'

inputs:

command: 'restore'

projects: '**/*.csproj'

- task: DotNetCoreCLI@2

displayName: 'Build'

inputs:

command: 'build'

projects: '**/*.csproj'

arguments: '--configuration \$(buildConfiguration)'

- task: DotNetCoreCLI@2

displayName: 'Publish'

inputs:

command: 'publish'

publishWebProjects: true

arguments: '--configuration \$(buildConfiguration) --output \$(Build.ArtifactStagingDirectory)'

- task: PublishBuildArtifacts@1

displayName: 'Publish Artifacts'

inputs:

pathToPublish: '\$(Build.ArtifactStagingDirectory)'

artifactName: 'drop'

Trigger Types

CI Triggers (Continuous Integration)

yaml

```
trigger:
  batch: true # Batch changes while build is running
  branches:
    include:
      - main
      - releases/*
  paths:
    include:
      - src/*
    exclude:
      - '**/*.md'
```

PR Triggers

yaml

```
pr:
  autoCancel: true # Cancel running builds on new push
  drafts: false # Don't trigger for draft PRs
  branches:
    include:
      - main
```

Scheduled Triggers

yaml

```
schedules:
  - cron: "0 3 * * Mon-Fri" # 3 AM UTC, weekdays
    displayName: Weekday build
    branches:
      include:
        - main
    batch: true
    always: false # Only if changes
```

Pipeline Triggers (Pipeline completion)

yaml

resources:

pipelines:

- pipeline: upstream

source: 'UpstreamPipeline'

trigger:

branches:

include:

- main

GitHub Actions Workflow Structure

yaml

name: CI/CD Pipeline

on:

push:

branches: [main, develop]

paths:

- 'src/**'

pull_request:

branches: [main]

schedule:

- cron: '0 0 * * *'

workflow_dispatch:

inputs:

environment:

description: 'Target environment'

required: true

default: 'development'

type: choice

options:

- development

- staging

- production

env:

DOTNET_VERSION: '8.0.x'

AZURE_WEBAPP_NAME: my-app

jobs:

build:

runs-on: ubuntu-latest

steps:

- **uses:** actions/checkout@v4

- **name:** Setup .NET

uses: actions/setup-dotnet@v4

with:

dotnet-version: \${ { env.DOTNET_VERSION } }

- **name:** Restore dependencies

run: dotnet restore

- **name:** Build

run: dotnet build --configuration Release --no-restore

- **name:** Test
run: dotnet test --no-build --verbosity normal

- **name:** Publish
run: dotnet publish -c Release -o ./publish

- **name:** Upload artifact
uses: actions/upload-artifact@v4
with:
 - name:** app
 - path:** ./publish

deploy:
needs: build
runs-on: ubuntu-latest
environment: production

steps:

- **name:** Download artifact
uses: actions/download-artifact@v4
with:
 - name:** app
- **name:** Deploy to Azure
uses: azure/webapps-deploy@v3
with:
 - app-name:** \${ env.AZURE_WEBAPP_NAME }
 - publish-profile:** \${ secrets.AZURE_WEBAPP_PUBLISH_PROFILE }

Pipeline Templates

Azure DevOps Templates

yaml

templates/build-template.yml

parameters:

- name: buildConfiguration
type: string
default: 'Release'
- name: projects
type: string
default: '**/*.csproj'

steps:

- task: DotNetCoreCLI@2
displayName: 'Restore'
inputs:
command: 'restore'
projects: \${{ parameters.projects }}
- task: DotNetCoreCLI@2
displayName: 'Build'
inputs:
command: 'build'
projects: \${{ parameters.projects }}
arguments: '--configuration \${{ parameters.buildConfiguration }}

yaml

main pipeline using template

stages:

- stage: Build
jobs:
- job: Build
steps:
- template: templates/build-template.yml
parameters:
buildConfiguration: 'Release'

Template Types

- **Step templates:** Reusable steps
- **Job templates:** Reusable jobs
- **Stage templates:** Reusable stages
- **Variable templates:** Reusable variable definitions
- **Extends templates:** Control what pipelines can do (security)

GitHub Reusable Workflows

```
yaml
# .github/workflows/reusable-build.yml
name: Reusable Build Workflow

on:
  workflow_call:
    inputs:
      environment:
        required: true
        type: string
    secrets:
      registry_password:
        required: true

jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: echo "Building for ${ inputs.environment }"
```

```
yaml
# Calling workflow
jobs:
  call-build:
    uses: ./.github/workflows/reusable-build.yml
    with:
      environment: production
    secrets:
      registry_password: ${ secrets.REGISTRY_PASSWORD }
```

Microsoft Learn: <https://learn.microsoft.com/en-us/azure/devops/pipelines/>

3.2 Design and Implement Pipeline Integration

Azure Integration Tasks

Azure Resource Manager Service Connection

- **Authentication methods:**

- Service Principal (automatic): Azure DevOps creates SP
- Service Principal (manual): Use existing SP
- Managed Identity: For self-hosted agents
- Workload Identity Federation: No secrets, OIDC-based (recommended)

yaml

Using service connection

- task: AzureCLI@2

inputs:

azureSubscription: 'my-service-connection'

scriptType: 'bash'

scriptLocation: 'inlineScript'

inlineScript: |

az account show

az group list

Common Azure Tasks

yaml

Azure App Service Deploy

- task: AzureWebApp@1

inputs:

azureSubscription: 'my-service-connection'

appType: 'webApp'

appName: 'my-app'

package: '\$(Pipeline.Workspace)/drop/*.zip'

Azure Kubernetes Service

- task: KubernetesManifest@1

inputs:

action: 'deploy'

kubernetesServiceConnection: 'my-aks-connection'

namespace: 'default'

manifests: '\$(Pipeline.Workspace)/manifests/*.yaml'

Azure Functions

- task: AzureFunctionApp@2

inputs:

azureSubscription: 'my-service-connection'

appType: 'functionApp'

appName: 'my-function-app'

package: '\$(Pipeline.Workspace)/drop/*.zip'

Azure Container Registry

- task: Docker@2

inputs:

containerRegistry: 'my-acr-connection'

repository: 'myapp'

command: 'buildAndPush'

Dockerfile: '**/Dockerfile'

tags: |

\$(Build.BuildId)

latest

Container Integration

Docker Build and Push

yaml

Azure Pipelines

```
- task: Docker@2
  displayName: 'Build and push image'
  inputs:
    containerRegistry: 'docker-hub-connection'
    repository: 'myorg/myapp'
    command: 'buildAndPush'
    Dockerfile: 'Dockerfile'
    buildContext: '.'
    tags: |
      $(Build.BuildId)
      $(Build.SourceBranchName)
      latest
```

yaml

GitHub Actions

```
- name: Login to Container Registry
  uses: docker/login-action@v3
  with:
    registry: ghcr.io
    username: ${ github.actor }
    password: ${ secrets.GITHUB_TOKEN }

- name: Build and push
  uses: docker/build-push-action@v5
  with:
    push: true
    tags: ghcr.io/${ github.repository }:${ github.sha }
```

Multi-stage Docker Build

dockerfile

Build stage

FROM mcr.microsoft.com/dotnet/sdk:8.0 AS build

WORKDIR /src

COPY *.csproj .

RUN dotnet restore

COPY . .

RUN dotnet publish -c Release -o /app/publish

Runtime stage

FROM mcr.microsoft.com/dotnet/aspnet:8.0 AS runtime

WORKDIR /app

COPY --from=build /app/publish .

EXPOSE 80

ENTRYPOINT ["dotnet", "MyApp.dll"]

Kubernetes Deployment

Helm Charts

yaml

Azure Pipelines Helm deployment

- task: HelmDeploy@0

inputs:

connectionType: 'Kubernetes Service Connection'

kubernetesServiceConnection: 'my-aks-connection'

namespace: 'production'

command: 'upgrade'

chartType: 'FilePath'

chartPath: '\$(Pipeline.Workspace)/charts/myapp'

releaseName: 'myapp'

overrideValues: 'image.tag=\$(Build.BuildId)'

install: true

waitForExecution: true

External Tool Integration

SonarQube/SonarCloud

yaml

```
- task: SonarQubePrepare@5
  inputs:
    SonarQube: 'sonar-connection'
    scannerMode: 'MSBuild'
    projectKey: 'my-project'
    projectName: 'My Project'

- task: DotNetCoreCLI@2
  inputs:
    command: 'build'

- task: SonarQubeAnalyze@5

- task: SonarQubePublish@5
  inputs:
    pollingTimeoutSec: '300'
```

WhiteSource/Mend

yaml

```
- task: WhiteSource@21
  inputs:
    cwd: '$(Build.SourcesDirectory)'
    projectName: 'my-project'
```

Microsoft Learn: <https://learn.microsoft.com/en-us/azure/devops/pipelines/tasks/reference/>

3.3 Design and Implement Pipeline Infrastructure

Agent Pools and Types

Microsoft-Hosted Agents

- **Available images:**
 - `ubuntu-latest` (Ubuntu 22.04)
 - `windows-latest` (Windows Server 2022)
 - `macos-latest` (macOS 14 Sonoma)
 - Specific versions: `ubuntu-22.04`, `windows-2019`, `macos-13`
- **Capabilities:** Pre-installed tools, auto-scaling, no maintenance
- **Limitations:**
 - 60-minute timeout (free tier), 6 hours (paid)
 - 10 GB storage
 - No persistence between jobs
 - Cannot access private networks directly

Self-Hosted Agents

- **Use cases:**
 - Access to private network resources
 - Specific hardware/software requirements
 - Longer build times needed
 - Cost optimization for high-volume builds
- **Setup:**

```
bash

# Download and configure agent
./config.sh --url https://dev.azure.com/yourorg --auth PAT --token <token> --pool <pool> --agent <agent>

# Run as service
sudo ./svc.sh install
sudo ./svc.sh start
```

- **Agent pools:** Organize agents by capability, environment, or team

Container Jobs

```
yaml
```

```
pool:
```

```
  vmImage: 'ubuntu-latest'
```

```
container: mcr.microsoft.com/dotnet/sdk:8.0
```

```
steps:
```

```
  - script: dotnet --version
```

Scale Set Agents

- **Auto-scaling:** Azure VM Scale Sets as agent pool
- **Configuration:** Min/max agents, idle timeout
- **Cost optimization:** Scale to zero when idle

Agent Selection

```
yaml
```

```
# Use specific pool
```

```
pool:
```

```
  name: 'MyPrivatePool'
```

```
  demands:
```

```
    - docker
```

```
    - Agent.OS -equals Linux
```

```
# Or use hosted agent
```

```
pool:
```

```
  vmImage: 'ubuntu-latest'
```

Pipeline Caching

yaml

NuGet cache

- task: Cache@2

inputs:

key: 'nuget | "\$(Agent.OS)" | **/packages.lock.json'

restoreKeys: |

nuget | "\$(Agent.OS)"

path: \$(NUGET_PACKAGES)

npm cache

- task: Cache@2

inputs:

key: 'npm | "\$(Agent.OS)" | package-lock.json'

restoreKeys: |

npm | "\$(Agent.OS)"

path: \$(npm_config_cache)

Docker layer cache

- task: Cache@2

inputs:

key: 'docker | "\$(Agent.OS)" | Dockerfile'

path: \$(Pipeline.Workspace)/docker-cache

restoreKeys: |

docker | "\$(Agent.OS)"

Parallel Jobs and Strategy

Matrix Strategy

yaml

```
jobs:
  - job: Build
    strategy:
      matrix:
        Linux:
          vmImage: 'ubuntu-latest'
        Windows:
          vmImage: 'windows-latest'
        macOS:
          vmImage: 'macos-latest'
      maxParallel: 3
    pool:
      vmImage: $(vmImage)
    steps:
      - script: echo Building on $(Agent.OS)
```

Parallel Deployment

yaml

```
strategy:
  parallel: 4
steps:
  - script: echo "Deploying chunk $SYSTEM_JOBPOSITIONINPHASE of $SYSTEM_TOTALJOBSINPHASE"
```

Microsoft Learn: <https://learn.microsoft.com/en-us/azure/devops/pipelines/agents/agents>

3.4 Design and Implement Deployments

Deployment Strategies

Rolling Deployment

- **Process:** Update instances in batches
- **Advantages:** Zero downtime, gradual rollout
- **Disadvantages:** Mixed versions during deployment

yaml

```
# Kubernetes rolling update
apiVersion: apps/v1
kind: Deployment
spec:
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 25%
      maxUnavailable: 25%
```

Blue-Green Deployment

- **Process:** Two identical environments, switch traffic
- **Advantages:** Instant rollback, zero downtime
- **Disadvantages:** Double infrastructure cost

yaml

```
# Azure App Service slots
- task: AzureWebApp@1
  inputs:
    azureSubscription: 'my-connection'
    appName: 'my-app'
    slotName: 'staging'
    package: '$(Pipeline.Workspace)/drop/*.zip'

- task: AzureAppServiceManage@0
  inputs:
    azureSubscription: 'my-connection'
    action: 'Swap Slots'
    webAppName: 'my-app'
    sourceSlot: 'staging'
    targetSlot: 'production'
```

Canary Deployment

- **Process:** Route small percentage of traffic to new version
- **Advantages:** Real user testing, controlled risk
- **Disadvantages:** Complexity, requires traffic management

yaml

```
# Azure Traffic Manager with weighted routing  
# or Application Gateway with path-based routing
```

Feature Flags

- **Tools:** Azure App Configuration, LaunchDarkly, Split.io
- **Use cases:**
 - Progressive rollout
 - A/B testing
 - Kill switch for features
 - User targeting

yaml

```
# Azure App Configuration feature flags  
- task: AzureAppConfiguration@7  
  inputs:  
    azureSubscription: 'my-connection'  
    configStoreEndpoint: 'https://my-config.azureconfig.io'  
    selectionMode: 'keyAndLabel'  
    keyAndLabel: '.appconfig.featureflag/*'
```

Deployment Jobs (Azure Pipelines)

yaml

stages:

- stage: Deploy

jobs:

- deployment: DeployWeb

displayName: 'Deploy to Production'

environment: 'production'

strategy:

runOnce:

deploy:

steps:

- download: current

artifact: drop

- task: AzureWebApp@1

inputs:

azureSubscription: 'my-connection'

appName: 'my-app'

package: '\$(Pipeline.Workspace)/drop/*.zip'

Environments

- **Purpose:** Logical target for deployment (dev, staging, prod)
- **Features:**
 - Deployment history
 - Approval gates
 - Resource tracking
 - Security policies

yaml

Environment with approvals

- deployment: Deploy

environment: 'production'

strategy:

runOnce:

deploy:

steps:

- script: echo Deploying

GitHub Environments

yaml

```
jobs:
  deploy:
    runs-on: ubuntu-latest
    environment:
      name: production
      url: https://myapp.azurewebsites.net
    steps:
      - run: echo "Deploying to production"
```

Microsoft Learn: <https://learn.microsoft.com/en-us/azure/devops/pipelines/process/deployment-jobs>

3.5 Design and Implement Release Management

Approvals and Checks

Pre-deployment Approvals

yaml

```
# Configure in Environment settings
# - Required approvers
# - Timeout
# - Instructions for approvers
```

Gates

- **Built-in gates:**
 - Invoke Azure Function
 - Query Azure Monitor alerts
 - Invoke REST API
 - Query work items
- **Use cases:**
 - Check incident status
 - Verify change management
 - Validate metrics

Branch Control

- Limit which branches can deploy to environment
- Require specific branch patterns

Business Hours

- Deploy only during specified hours
- Configure time zone

Release Orchestration

Classic Release Pipelines

- **Artifacts:** Build artifacts, Git repos, Jenkins, etc.
- **Stages:** Visual pipeline editor
- **Triggers:** Continuous deployment, scheduled, manual
- **Variables:** Stage-scoped, release-scoped

YAML Multi-Stage Pipelines

```
yaml

stages:
  - stage: Build
    jobs:
      - job: Build
        steps:
          - script: echo Building

  - stage: DeployDev
    dependsOn: Build
    condition: succeeded()
    jobs:
      - deployment: DeployDev
        environment: 'dev'

  - stage: DeployProd
    dependsOn: DeployDev
    condition: and(succeeded(), eq(variables['Build.SourceBranch'], 'refs/heads/main'))
    jobs:
      - deployment: DeployProd
        environment: 'production'
```

Conditions and Dependencies

yaml

Stage conditions

stages:

- stage: A

- jobs:

- job: A1

- steps:

- script: echo A1

- stage: B

- dependsOn: A

- condition: succeeded()

- stage: C

- dependsOn:

- A

- B

- condition: |

- and(

- eq(dependencies.A.result, 'Succeeded'),

- eq(dependencies.B.result, 'Succeeded'),

- eq(variables['Build.SourceBranch'], 'refs/heads/main')

-)

Artifact Management

Pipeline Artifacts (Recommended)

yaml

Publish

- **task:** PublishPipelineArtifact@1

inputs:

targetPath: '\$(Build.ArtifactStagingDirectory)'

artifact: 'drop'

Download in same pipeline

- **task:** DownloadPipelineArtifact@2

inputs:

artifact: 'drop'

path: '\$(Pipeline.Workspace)/drop'

Download from another pipeline

- **task:** DownloadPipelineArtifact@2

inputs:

buildType: 'specific'

project: 'MyProject'

definition: 'MyPipeline'

artifact: 'drop'

Azure Artifacts

- **Feed types:** NuGet, npm, Maven, Python, Universal Packages
- **Views:** @Local, @Prerelease, @Release
- **Upstream sources:** nuget.org, npmjs.com, Maven Central, PyPI

yaml

Publish to Azure Artifacts

- **task:** NuGetCommand@2

inputs:

command: 'push'

packagesToPush: '\$(Build.ArtifactStagingDirectory)/**/*.nupkg'

nuGetFeedType: 'internal'

publishVstsFeed: 'my-feed'

Microsoft Learn: <https://learn.microsoft.com/en-us/azure/devops/pipelines/release/>

3.6 Design and Implement Testing Strategy

Test Types in Pipelines

Unit Tests

yaml

.NET

- task: DotNetCoreCLI@2

inputs:

command: 'test'

projects: '**/*Tests/*.csproj'

arguments: '--configuration \$(buildConfiguration) --collect:"XPlat Code Coverage"

Node.js

- script: npm test -- --coverage

displayName: 'Run tests'

Integration Tests

yaml

Run against deployed environment

- task: DotNetCoreCLI@2

inputs:

command: 'test'

projects: '**/IntegrationTests.csproj'

env:

API_URL: \$(DeployedApiUrl)

UI/End-to-End Tests

yaml

Playwright

- task: NodeTool@0

inputs:

versionSpec: '18.x'

- script: |

npm ci

npx playwright install --with-deps

npx playwright test

displayName: 'Run Playwright tests'

- task: PublishTestResults@2

inputs:

testResultsFormat: 'JUnit'

testResultsFiles: 'test-results/results.xml'

Load Tests

yaml

Azure Load Testing

- **task:** AzureLoadTest@1

inputs:

azureSubscription: 'my-connection'

loadTestConfigFile: 'loadtest.yaml'

resourceGroup: 'my-rg'

loadTestResource: 'my-load-test'

Code Coverage

yaml

Publish code coverage

- **task:** PublishCodeCoverageResults@2

inputs:

summaryFileLocation: '\$(Agent.TempDirectory)/**/*.cobertura.xml'

failIfCoverageEmpty: true

Enforce minimum coverage

- **script:** |

COVERAGE=\$(cat coverage.json | jq '.total.lines.pct')

if ((\$(echo "\$COVERAGE < 80" | bc -l))); then

echo "Coverage \$COVERAGE% is below 80%"

exit 1

fi

Test Results Publishing

yaml

- **task:** PublishTestResults@2

inputs:

testResultsFormat: 'VSTest' # or 'JUnit', 'NUnit', 'xUnit'

testResultsFiles: '**/*.trx'

mergeTestResults: true

failTaskOnFailedTests: true

Flaky Test Management

- **Detection:** Azure DevOps automatically detects flaky tests
- **Configuration:** Project Settings → Pipelines → Test Management
- **Resolution tracking:** Mark tests as flaky, track fix progress

Microsoft Learn: <https://learn.microsoft.com/en-us/azure/devops/pipelines/ecosystems/>

3.7 Design and Implement Infrastructure as Code (IaC)

Azure Resource Manager (ARM) Templates

json

```
{
  "$schema": "https://schema.management.azure.com/schemas/2019-04-01/deploymentTemplate.json",
  "contentVersion": "1.0.0.0",
  "parameters": {
    "webAppName": {
      "type": "string",
      "metadata": {
        "description": "Name of the web app"
      }
    },
    "sku": {
      "type": "string",
      "defaultValue": "S1",
      "allowedValues": ["F1", "D1", "B1", "B2", "S1", "S2", "P1", "P2"]
    }
  },
  "variables": {
    "appServicePlanName": "[concat(parameters('webAppName'), '-plan')]"
  },
  "resources": [
    {
      "type": "Microsoft.Web/serverfarms",
      "apiVersion": "2022-03-01",
      "name": "[variables('appServicePlanName')]",
      "location": "[resourceGroup().location]",
      "sku": {
        "name": "[parameters('sku')]"
      }
    },
    {
      "type": "Microsoft.Web/sites",
      "apiVersion": "2022-03-01",
      "name": "[parameters('webAppName')]",
      "location": "[resourceGroup().location]",
      "dependsOn": [
        "[resourceId('Microsoft.Web/serverfarms', variables('appServicePlanName'))]"
      ],
      "properties": {
        "serverFarmId": "[resourceId('Microsoft.Web/serverfarms', variables('appServicePlanName'))]"
      }
    }
  ],
  "outputs": {
    "webAppUrl": {
      "type": "string",
      "value": "[uri('https://', parameters('webAppName'), '.azurewebsites.net')]"
    }
  }
}
```

```

    "type": "string",
    "value": "[concat('https://', reference(parameters('webAppName')).defaultHostName)]"
  }
}
}

```

Bicep (Recommended for Azure)

```

bicep

// main.bicep
param webAppName string
param location string = resourceGroup().location
param sku string = 'S1'

var appServicePlanName = '${webAppName}-plan'

resource appServicePlan 'Microsoft.Web/serverfarms@2022-03-01' = {
  name: appServicePlanName
  location: location
  sku: {
    name: sku
  }
}

resource webApp 'Microsoft.Web/sites@2022-03-01' = {
  name: webAppName
  location: location
  properties: {
    serverFarmId: appServicePlan.id
  }
}

output webAppUrl string = 'https://${webApp.properties.defaultHostName}'

```

yaml

```
# Deploy Bicep
- task: AzureCLI@2
  inputs:
    azureSubscription: 'my-connection'
    scriptType: 'bash'
    scriptLocation: 'inlineScript'
    inlineScript: |
      az deployment group create \
        --resource-group $(resourceGroup) \
        --template-file main.bicep \
        --parameters webAppName=$(webAppName)
```

Terraform

hcl

main.tf

terraform {

required_providers {

azurerm = {

source = "hashicorp/azurerm"

version = "~> 3.0"

}

}

backend "azurerm" {

resource_group_name = "tfstate-rg"

storage_account_name = "tfstatestorage"

container_name = "tfstate"

key = "terraform.tfstate"

}

}

provider "azurerm" {

features {}

}

variable "web_app_name" {

type = string

}

variable "location" {

type = string

default = "eastus"

}

resource "azurerm_resource_group" "main" {

name = "\${var.web_app_name}-rg"

location = var.location

}

resource "azurerm_service_plan" "main" {

name = "\${var.web_app_name}-plan"

resource_group_name = azurerm_resource_group.main.name

location = azurerm_resource_group.main.location

os_type = "Linux"

sku_name = "S1"

}

resource "azurerm_linux_web_app" "main" {

name = "\${var.web_app_name}"

```

name = var.web_app_name
resource_group_name = azurerm_resource_group.main.name
location           = azurerm_resource_group.main.location
service_plan_id    = azurerm_service_plan.main.id

site_config {
  always_on = true
}
}

output "web_app_url" {
  value = "https://${azurerm_linux_web_app.main.default_hostname}"
}

```

yaml

```

# Terraform pipeline
- task: TerraformInstaller@1
  inputs:
    terraformVersion: 'latest'

- task: TerraformTaskV4@4
  inputs:
    provider: 'azurerm'
    command: 'init'
    backendServiceArm: 'my-connection'
    backendAzureRmResourceGroupName: 'tfstate-rg'
    backendAzureRmStorageAccountName: 'tfstatestorage'
    backendAzureRmContainerName: 'tfstate'
    backendAzureRmKey: 'terraform.tfstate'

- task: TerraformTaskV4@4
  inputs:
    provider: 'azurerm'
    command: 'plan'
    environmentServiceNameAzureRM: 'my-connection'

- task: TerraformTaskV4@4
  inputs:
    provider: 'azurerm'
    command: 'apply'
    environmentServiceNameAzureRM: 'my-connection'

```

Azure CLI in Pipelines

yaml

```
- task: AzureCLI@2
  inputs:
    azureSubscription: 'my-connection'
    scriptType: 'bash'
    scriptLocation: 'inlineScript'
    inlineScript: |
      # Create resource group
      az group create --name myRG --location eastus

      # Create App Service
      az webapp create \
        --resource-group myRG \
        --plan myPlan \
        --name myApp \
        --runtime "DOTNET|8.0"
```

Azure PowerShell in Pipelines

yaml

```
- task: AzurePowerShell@5
  inputs:
    azureSubscription: 'my-connection'
    ScriptType: 'InlineScript'
    Inline: |
      New-AzResourceGroup -Name "myRG" -Location "eastus"
      New-AzWebApp -ResourceGroupName "myRG" -Name "myApp" -AppServicePlan "myPlan"
    azurePowerShellVersion: 'LatestVersion'
```

Microsoft Learn: <https://learn.microsoft.com/en-us/azure/azure-resource-manager/bicep/>

Domain 4: Develop a Security and Compliance Plan (10-15%)

4.1 Design and Implement Secure Access to Pipeline Resources

Service Connections Security

Service Connection Types

- **Azure Resource Manager:** Access Azure resources
- **Docker Registry:** Container registries
- **Kubernetes:** AKS and other clusters
- **Generic:** REST APIs, SSH, etc.
- **GitHub/Azure Repos:** Source code access

Security Best Practices

- **Workload Identity Federation:** Preferred for Azure (no secrets)
- **Least privilege:** Grant minimum required permissions
- **Scope limitation:** Restrict to specific subscriptions/resource groups
- **Pipeline authorization:** Require pipeline approval for use

yaml

Service connection with limited scope

- **task:** AzureCLI@2

inputs:

azureSubscription: 'limited-scope-connection' *# Only has access to specific RG*

scriptType: 'bash'

scriptLocation: 'inlineScript'

inlineScript: |

az webapp deploy --resource-group allowed-rg --name my-app

Pipeline Permissions

Pipeline-Level Permissions

- **Build:** Queue builds
- **Edit:** Modify pipeline definition
- **Delete:** Remove pipeline
- **Manage build resources:** Configure retention
- **View builds:** Read-only access

Resource Authorization

yaml

Resource must be authorized for pipeline

resources:

repositories:

- repository: templates

type: git

name: MyProject/PipelineTemplates

containers:

- container: mycontainer

image: myregistry.azurecr.io/myimage:latest

endpoint: my-acr-connection

Agent Security

Self-Hosted Agent Security

- **Run as non-root:** Don't run agent as administrator
- **Network isolation:** Use private networks
- **Ephemeral agents:** Recreate after each job
- **Dedicated agents:** Separate pools for sensitive workloads

Interactive vs. Service Mode

- **Service mode:** Runs as background service (recommended)
- **Interactive mode:** Runs in foreground (for debugging)

yaml

Use specific pool for sensitive deployments

pool:

name: 'SecurePool'

demands:

- Agent.ComputerName -equals SecureAgent1

GitHub Actions Security

Secrets Management

yaml

```
jobs:
  deploy:
    runs-on: ubuntu-latest
    steps:
      - name: Deploy
        env:
          AZURE_CREDENTIALS: ${{ secrets.AZURE_CREDENTIALS }}
        run: |
          echo "$AZURE_CREDENTIALS" | az login --service-principal -u <id> -p /dev/stdin --tenant <t>
```

OIDC Authentication (Recommended)

yaml

```
jobs:
  deploy:
    runs-on: ubuntu-latest
    permissions:
      id-token: write
      contents: read
    steps:
      - name: Azure Login
        uses: azure/login@v2
        with:
          client-id: ${{ secrets.AZURE_CLIENT_ID }}
          tenant-id: ${{ secrets.AZURE_TENANT_ID }}
          subscription-id: ${{ secrets.AZURE_SUBSCRIPTION_ID }}
```

Environment Protection Rules

- **Required reviewers:** Manual approval before deployment
- **Wait timer:** Delay before deployment
- **Deployment branches:** Limit which branches can deploy

Microsoft Learn: <https://learn.microsoft.com/en-us/azure/devops/pipelines/library/service-endpoints>

4.2 Design and Implement Secret Management

Azure Key Vault Integration

Variable Groups Linked to Key Vault

yaml

variables:

- **group:** 'keyvault-linked-group'

steps:

- **script:** |
 echo "Using secret: \$(mySecret)"
 displayName: 'Use secret'

Key Vault Task

yaml

- **task:** AzureKeyVault@2

inputs:

azureSubscription: 'my-connection'

KeyVaultName: 'my-keyvault'

SecretsFilter: 'secret1,secret2' *# or * for all*

RunAsPreJob: true

Key Vault References in ARM/Bicep

json

```
{
  "parameters": {
    "adminPassword": {
      "reference": {
        "keyVault": {
          "id": "/subscriptions/.../resourceGroups/.../providers/Microsoft.KeyVault/vaults/myvault"
        },
        "secretName": "vmAdminPassword"
      }
    }
  }
}
```

Pipeline Secrets

Azure DevOps Variables

- **Secret variables:** Masked in logs, encrypted at rest
- **Variable groups:** Shared across pipelines
- **Library:** Secure files and variable groups

yaml

variables:

- **name:** publicVar
value: 'visible'
- **name:** secretVar
value: \$(MySecretFromLibrary) *# From variable group*

steps:

- **script:** |
 echo \$(publicVar) *# Prints: visible*
 echo \$(secretVar) *# Prints: ****

Secure Files

yaml

- **task:** DownloadSecureFile@1
name: myCert
inputs:
 secureFile: 'my-certificate.pfx'
- **script:** |
 echo "Certificate path: \$(myCert.secureFilePath)"

GitHub Secrets

Organization, Repository, and Environment Secrets

yaml

Access at different scopes

jobs:

build:

runs-on: ubuntu-latest

environment: production

steps:

- **run:** echo \${ secrets.REPO_SECRET } *# Repository secret*
- **run:** echo \${ secrets.ORG_SECRET } *# Organization secret*
- **run:** echo \${ secrets.ENV_SECRET } *# Environment secret*

Secret Masking

- Secrets automatically masked in logs
- Use `::add-mask::` for dynamic values:

yaml

```
- run: |  
  DYNAMIC_SECRET=$(generate-secret)  
  echo "::add-mask::$DYNAMIC_SECRET"  
  echo "Using secret: $DYNAMIC_SECRET"
```

Secret Rotation

- **Automated rotation:** Azure Key Vault rotation policies
- **Manual process:** Update in Key Vault, pipelines pick up automatically
- **Zero-downtime rotation:** Keep old version active during transition

Microsoft Learn: <https://learn.microsoft.com/en-us/azure/devops/pipelines/release/azure-key-vault>

4.3 Design and Implement Pipeline Security

Secure Pipeline Patterns

Extends Templates (Security Baseline)

yaml

```
# templates/secure-pipeline.yml  
parameters:  
  - name: buildSteps  
    type: stepList  
    default: []  
  
stages:  
  - stage: Build  
    jobs:  
      - job: SecureBuild  
        steps:  
          # Security scans always run first  
          - task: CredScan@3  
  
          # Then custom build steps  
          - ${{ parameters.buildSteps }}  
  
          # Security validation at end  
          - task: PostAnalysis@2
```

```
yaml
```

```
# Consumer pipeline
```

```
extends:
```

```
  template: templates/secure-pipeline.yml
```

```
parameters:
```

```
  buildSteps:
```

```
    - script: npm run build
```

Protected Resources

- **Environments:** Require approvals
- **Service connections:** Require authorization
- **Agent pools:** Limit access
- **Variable groups:** Restrict who can use
- **Secure files:** Controlled distribution

Pipeline Permissions Best Practices

- **Project Collection Administrators:** Full control
- **Project Administrators:** Project-level control
- **Build Administrators:** Manage pipelines
- **Contributors:** Queue/view builds
- **Readers:** View only

Container Security

```
yaml
```

```
# Use specific image tags, not 'latest'
```

```
container:
```

```
  image: mcr.microsoft.com/dotnet/sdk:8.0.100-alpine3.18
```

```
# Or use digest for immutability
```

```
container:
```

```
  image: myregistry.azurecr.io/myimage@sha256:abc123...
```

Branch Protection for Pipelines

yaml

Require PR for pipeline changes

trigger:

branches:

include:

- main

paths:

include:

- azure-pipelines.yml

- .azure-pipelines/*

Microsoft Learn: <https://learn.microsoft.com/en-us/azure/devops/pipelines/security/overview>

4.4 Design and Implement Security Scanning

Static Application Security Testing (SAST)

Microsoft Security DevOps

yaml

- **task:** MicrosoftSecurityDevOps@1

displayName: 'Run Microsoft Security DevOps'

inputs:

categories: 'secrets,code'

SonarQube/SonarCloud Security

yaml

- **task:** SonarQubePrepare@5

inputs:

SonarQube: 'sonar-connection'

scannerMode: 'MSBuild'

projectKey: 'my-project'

extraProperties: |

sonar.security.hotspots.inheritedRules=true

GitHub Code Scanning

yaml

```
# .github/workflows/codeql.yml
```

```
name: CodeQL
```

```
on:
```

```
  push:
```

```
    branches: [main]
```

```
  pull_request:
```

```
    branches: [main]
```

```
  schedule:
```

```
    - cron: '0 0 * * 0'
```

```
jobs:
```

```
  analyze:
```

```
    runs-on: ubuntu-latest
```

```
    permissions:
```

```
      actions: read
```

```
      contents: read
```

```
      security-events: write
```

```
    strategy:
```

```
      matrix:
```

```
        language: ['csharp', 'javascript']
```

```
  steps:
```

```
    - uses: actions/checkout@v4
```

```
    - name: Initialize CodeQL
```

```
      uses: github/codeql-action/init@v3
```

```
      with:
```

```
        languages: ${{ matrix.language }}
```

```
    - name: Build
```

```
      run: dotnet build
```

```
    - name: Perform CodeQL Analysis
```

```
      uses: github/codeql-action/analyze@v3
```

Software Composition Analysis (SCA)

Dependency Scanning

yaml

Azure Pipelines - WhiteSource/Mend

- **task:** WhiteSource@21

inputs:

cwd: '\$(Build.SourcesDirectory)'

projectName: 'my-project'

GitHub - Dependabot

.github/dependabot.yml

version: 2

updates:

- **package-ecosystem:** "npm"

directory: "/"

schedule:

interval: "weekly"

open-pull-requests-limit: 10

OWASP Dependency Check

yaml

- **task:** dependency-check-build-task@6

inputs:

projectName: 'my-project'

scanPath: '\$(Build.SourcesDirectory)'

format: 'HTML,JUNIT'

failOnCVSS: '7'

Container Security Scanning

Trivy

yaml

- **script:** |

trivy image --severity HIGH,CRITICAL --exit-code 1 myimage:\$(Build.BuildId)

displayName: 'Scan container image'

Microsoft Defender for Containers

- Automatic scanning in Azure Container Registry
- Runtime protection in AKS
- Vulnerability assessment

Secret Scanning

Git Credential Scanner

```
yaml
- task: CredScan@3
  inputs:
    outputFormat: 'pre'
    suppressionsFile: 'suppressions.json'

- task: PostAnalysis@2
  inputs:
    CredScan: true
```

GitHub Secret Scanning

- Automatic detection of known secret patterns
- Custom patterns for organization-specific secrets
- Push protection to prevent commits with secrets

DAST (Dynamic Application Security Testing)

```
yaml
# OWASP ZAP
- task: owaspzap@1
  inputs:
    aggressivemode: false
    scantype: 'targetedScan'
    threshold: 'Medium'
    targeturl: 'https://$(DeployedAppUrl)'
```

Microsoft Learn: <https://learn.microsoft.com/en-us/azure/devops/pipelines/security/overview>

4.5 Design and Implement Governance and Compliance

Azure Policy for DevOps

Policy Definitions

- Enforce tagging requirements
- Restrict resource types
- Require specific configurations
- Audit compliance

json

```
{
  "policyRule": {
    "if": {
      "allOf": [
        {
          "field": "type",
          "equals": "Microsoft.Web/sites"
        },
        {
          "field": "Microsoft.Web/sites/httpsOnly",
          "notEquals": "true"
        }
      ]
    },
    "then": {
      "effect": "deny"
    }
  }
}
```

Pipeline Compliance

Required Templates

yaml

```
# Organization-mandated template
extends:
  template: templates/compliance-checks.yml@security-templates
parameters:
  scanType: 'full'
```

Audit Logs

- Pipeline run history
- Permission changes
- Service connection usage
- Variable group access

Open Source Compliance

License Scanning

yaml

```
# Check for prohibited licenses
- task: WhiteSource@21
  inputs:
    projectName: 'my-project'
  # Fails if GPL or other restricted licenses found
```

Software Bill of Materials (SBOM)

yaml

```
# Generate SBOM
- task: ManifestGeneratorTask@0
  inputs:
    BuildDropPath: '$(Build.ArtifactStagingDirectory)'
    PackageName: 'my-package'
    PackageVersion: '$(Build.BuildNumber)'
```

Change Management Integration

ServiceNow Integration

yaml

```
- task: ServiceNow-DevOps-Agent-Config-Setup@1
  inputs:
    connectedServiceName: 'servicenow-connection'
    toolId: 'my-tool'

- task: ServiceNow-DevOps-Agent-Artifact-Registration@1
  inputs:
    connectedServiceName: 'servicenow-connection'
    artifactsPayload: |
      {
        "name": "my-artifact",
        "version": "$(Build.BuildNumber)"
      }
```

Evidence Collection

- Automated test results
- Security scan reports
- Approval audit trails
- Deployment logs
- Infrastructure state

Microsoft Learn: <https://learn.microsoft.com/en-us/azure/governance/policy/>

Domain 5: Implement an Instrumentation Strategy (10-15%)

5.1 Configure Monitoring for DevOps

Azure Monitor Overview

Components

- **Metrics:** Numerical data collected at intervals
- **Logs:** Detailed records in Log Analytics
- **Alerts:** Notifications based on conditions
- **Dashboards:** Visualizations
- **Workbooks:** Interactive reports

Application Insights

Setup

```
yaml

# Add Application Insights during deployment
- task: AzureCLI@2
  inputs:
    azureSubscription: 'my-connection'
    scriptType: 'bash'
    scriptLocation: 'inlineScript'
    inlineScript: |
      az webapp config appsettings set \
        --resource-group myRG \
        --name myApp \
        --settings APPLICATIONINSIGHTS_CONNECTION_STRING=$(AppInsightsConnectionString)
```

Key Metrics

- **Availability:** Uptime monitoring
- **Performance:** Response times, dependencies
- **Failures:** Exceptions, failed requests
- **Usage:** Users, sessions, page views
- **Custom events:** Business metrics

Kusto Query Language (KQL)

```
kusto

// Failed requests in last hour
requests
| where timestamp > ago(1h)
| where success == false
| summarize count() by resultCode
| order by count_desc

// Slow dependencies
dependencies
| where timestamp > ago(1h)
| where duration > 1000
| summarize count() by name, type
| order by count_desc

// Exception trends
exceptions
| where timestamp > ago(24h)
| summarize count() by bin(timestamp, 1h), type
| render timechart
```

Azure Monitor Alerts

Alert Types

- **Metric alerts:** Threshold-based (CPU > 80%)
- **Log alerts:** KQL query results
- **Activity log alerts:** Azure operations
- **Smart detection:** AI-based anomalies

Alert Actions

- **Action groups:** Email, SMS, webhook, Logic App, Azure Function
- **ITSM integration:** ServiceNow, Cherwell
- **Automation:** Azure Automation runbook

yaml

Create alert rule via CLI

- **task:** AzureCLI@2

inputs:

azureSubscription: 'my-connection'

scriptType: 'bash'

scriptLocation: 'inlineScript'

inlineScript: |

az monitor metrics alert create \

--name "High CPU Alert" \

--resource-group myRG \

--scopes "/subscriptions/.../resourceGroups/myRG/providers/Microsoft.Web/sites/myApp" \

--condition "avg Percentage CPU > 80" \

--window-size 5m \

--evaluation-frequency 1m \

--action "/subscriptions/.../resourceGroups/myRG/providers/microsoft.insights/actionGroups/m

Microsoft Learn: <https://learn.microsoft.com/en-us/azure/azure-monitor/>

5.2 Configure Logging

Azure Monitor Logs

Log Analytics Workspace

- Central repository for logs
- Data retention configuration
- Access control
- Cross-resource queries

Data Sources

- **Azure resources:** Diagnostic settings
- **Virtual machines:** Log Analytics agent
- **Containers:** Container Insights
- **Applications:** Application Insights
- **Custom:** REST API, Logic Apps

Diagnostic Settings

yaml

- task: AzureCLI@2

inputs:

azureSubscription: 'my-connection'

scriptType: 'bash'

scriptLocation: 'inlineScript'

inlineScript: |

az monitor diagnostic-settings create \

--name "send-to-log-analytics" \

--resource "/subscriptions/.../resourceGroups/myRG/providers/Microsoft.Web/sites/myApp" \

--workspace "/subscriptions/.../resourceGroups/myRG/providers/Microsoft.OperationalInsights" \

--logs '["category":"AppServiceHTTPLogs","enabled":true],["category":"AppServiceConsoleLogs","enabled":true]' \

--metrics '["category":"AllMetrics","enabled":true]'

Distributed Tracing

End-to-End Correlation

- **Correlation ID:** Tracks requests across services
- **Operation ID:** Groups related telemetry
- **Parent ID:** Links child operations

Application Map

- Visual representation of dependencies
- Performance metrics per component
- Failure rates

Container Logging

Container Insights

yaml

Enable Container Insights

- task: AzureCLI@2

inputs:

azureSubscription: 'my-connection'

scriptType: 'bash'

scriptLocation: 'inlineScript'

inlineScript: |

az aks enable-addons \

--resource-group myRG \

--name myAKS \

--addons monitoring \

--workspace-resource-id "/subscriptions/.../resourceGroups/myRG/providers/Microsoft.Operati

Container Logs Query

kusto

ContainerLog

| where TimeGenerated > ago(1h)

| where LogEntry contains "error"

| project TimeGenerated, ContainerName, LogEntry

| order by TimeGenerated desc

Microsoft Learn: <https://learn.microsoft.com/en-us/azure/azure-monitor/logs/>

5.3 Configure Pipeline Analytics

Azure DevOps Analytics

Pipeline Reports

- **Pass rate:** Historical success/failure
- **Duration:** Build/release times
- **Test results:** Test pass rates
- **Code coverage:** Trends over time

Analytics Views

- Built-in views for work items
- Custom views for specific queries
- Power BI integration

Pipeline Telemetry

Build Metrics

```
yaml

# Log custom metrics
- script: |
    echo "##vso[task.setvariable variable=BuildDuration]$(expr $(date +%s) - $(BUILD_START_TIMESTAMP))"
    echo "##vso[task.setvariable variable=TestCount]$(cat test-results.json | jq '.total')"
```

Deployment Frequency

- Track deployments per environment
- Measure lead time for changes
- Calculate deployment failure rate
- Monitor mean time to recovery (MTTR)

DORA Metrics

- **Deployment Frequency:** How often code is deployed
- **Lead Time for Changes:** Time from commit to production
- **Change Failure Rate:** Percentage of deployments causing failures
- **Time to Restore Service:** Time to recover from failures

GitHub Insights

- **Actions usage:** Minutes consumed, run durations
- **Repository insights:** Commits, PRs, issues
- **Dependency insights:** Vulnerability alerts, updates

Microsoft Learn: <https://learn.microsoft.com/en-us/azure/devops/report/dashboards/>

5.4 Configure Alerting

Alert Strategy

Alert Severity

- **Sev 0 (Critical):** Immediate response required
- **Sev 1 (High):** Urgent attention needed
- **Sev 2 (Medium):** Should be addressed soon
- **Sev 3 (Low):** Informational

Alert Fatigue Prevention

- Set appropriate thresholds
- Use time-based conditions
- Implement alert suppression
- Regular alert review and tuning

Pipeline Alerts

Build Failure Notifications

```
yaml

# Azure DevOps - via service hooks or subscriptions
# Configure in Project Settings → Notifications

# GitHub Actions
- name: Notify on failure
  if: failure()
  uses: 8398a7/action-slack@v3
  with:
    status: failure
    fields: repo,message,commit,author,action,eventName,ref,workflow
  env:
    SLACK_WEBHOOK_URL: ${ secrets.SLACK_WEBHOOK }
```

Deployment Alerts

```
yaml

- task: AzureCLI@2
  inputs:
    azureSubscription: 'my-connection'
    scriptType: 'bash'
    scriptLocation: 'inlineScript'
    inlineScript: |
      # Create deployment annotation
      az monitor activity-log alert create \
        --name "Deployment Alert" \
        --resource-group myRG \
        --condition category=Administrative and operationName="Microsoft.Web/sites/write" \
        --action-group myActionGroup
```

Health Monitoring

Availability Tests

yaml

Create availability test

- **task:** AzureCLI@2

inputs:

azureSubscription: 'my-connection'

scriptType: 'bash'

scriptLocation: 'inlineScript'

inlineScript: |

az monitor app-insights web-test create \

--resource-group myRG \

--app-insights myAppInsights \

--name "Homepage Availability" \

--location "East US" \

--web-test-kind "ping" \

--frequency 300 \

--timeout 120 \

--defined-web-test-name "Homepage Availability" \

--synthetic-monitor-id "homepage-test" \

--web-test "<WebTest Name='Homepage' Enabled='True'><Items><Request Method='GET'

Service Health Alerts

- Azure Service Health for platform issues
- Resource Health for specific resources
- Planned maintenance notifications

Microsoft Learn: <https://learn.microsoft.com/en-us/azure/azure-monitor/alerts/>

Quick Reference: Essential Commands

Azure CLI

```
bash
```

Login

```
az login
```

```
az account set --subscription "My Subscription"
```

Resource Groups

```
az group create --name myRG --location eastus
```

```
az group delete --name myRG --yes --no-wait
```

App Service

```
az webapp create --resource-group myRG --plan myPlan --name myApp --runtime "DOTNET|8.0"
```

```
az webapp deployment slot create --resource-group myRG --name myApp --slot staging
```

```
az webapp deployment slot swap --resource-group myRG --name myApp --slot staging --target-slot p
```

Container Registry

```
az acr create --resource-group myRG --name myACR --sku Standard
```

```
az acr login --name myACR
```

```
az acr build --registry myACR --image myimage:v1 .
```

AKS

```
az aks create --resource-group myRG --name myAKS --node-count 3
```

```
az aks get-credentials --resource-group myRG --name myAKS
```

```
az aks enable-addons --resource-group myRG --name myAKS --addons monitoring
```

Key Vault

```
az keyvault create --name myKV --resource-group myRG --location eastus
```

```
az keyvault secret set set --vault-name myKV --name mySecret --value "secretvalue"
```

```
az keyvault secret show --vault-name myKV --name mySecret --query value -o tsv
```

Azure DevOps CLI

bash

Configure

```
az devops configure --defaults organization=https://dev.azure.com/myorg project=myproject
```

Pipelines

```
az pipelines create --name "My Pipeline" --yaml-path azure-pipelines.yml --repository myrepo --branch main
```

```
az pipelines run --name "My Pipeline"
```

```
az pipelines build list --top 10
```

Repos

```
az repos list
```

```
az repos policy list --repository-id <repo-id>
```

```
az repos pr create --source-branch feature/my-feature --target-branch main --title "My PR"
```

Variable Groups

```
az pipelines variable-group create --name "my-vars" --variables var1=value1 var2=value2
```

```
az pipelines variable-group variable create --group-id <id> --name secretVar --value "secret" --secret
```

Git Commands

bash

Branching

git checkout -b feature/my-feature

git push -u origin feature/my-feature

git branch -d feature/my-feature

git push origin --delete feature/my-feature

Rebasing

git fetch origin

git rebase origin/main

git rebase -i HEAD~3 # *Interactive rebase*

Cherry-pick

git cherry-pick <commit-hash>

Stashing

git stash

git stash pop

git stash list

git stash drop

History cleanup

git rebase -i HEAD~5 # *Squash commits*

git commit --amend # *Modify last commit*

LFS

git lfs track "*.psd"

git lfs ls-files

git lfs fetch --all

Kubernetes Commands

```
bash
```

Context

```
kubectl config get-contexts
```

```
kubectl config use-context my-context
```

Deployments

```
kubectl apply -f deployment.yaml
```

```
kubectl rollout status deployment/myapp
```

```
kubectl rollout undo deployment/myapp
```

```
kubectl scale deployment/myapp --replicas=5
```

Pods

```
kubectl get pods -n mynamespace
```

```
kubectl logs pod/mypod -f
```

```
kubectl exec -it pod/mypod -- /bin/bash
```

```
kubectl describe pod/mypod
```

Services

```
kubectl get svc
```

```
kubectl expose deployment/myapp --port=80 --target-port=8080
```

ConfigMaps and Secrets

```
kubectl create configmap myconfig --from-literal=key=value
```

```
kubectl create secret generic mysecret --from-literal=password=secret
```

```
kubectl get secret mysecret -o jsonpath='{.data.password}' | base64 -d
```

Docker Commands


```
bash
```

```
# Build
```

```
docker build -t myimage:v1 .
```

```
docker build -t myimage:v1 --no-cache .
```

```
docker build -t myimage:v1 -f Dockerfile.prod .
```

```
# Run
```

```
docker run -d -p 8080:80 myimage:v1
```

```
docker run -it --rm myimage:v1 /bin/bash
```

```
# Push
```

```
docker tag myimage:v1 myregistry.azurecr.io/myimage:v1
```

```
docker push myregistry.azurecr.io/myimage:v1
```

```
# Inspect
```

```
docker images
```

```
docker ps -a
```

```
docker logs container_id
```

```
docker inspect container_id
```

Key Exam Tips

What to Focus On

1. **YAML Pipelines:** Know syntax, triggers, conditions, templates
2. **Service Connections:** Types, security, authorization
3. **Branching Strategies:** GitFlow, trunk-based, GitHub Flow
4. **Deployment Strategies:** Blue-green, canary, rolling
5. **Security:** Key Vault, secret management, scanning tools
6. **Monitoring:** Application Insights, KQL queries, alerts
7. **IaC:** Bicep, ARM, Terraform basics
8. **Containers:** Docker, ACR, AKS deployments

Common Exam Scenarios

- Choosing the right branching strategy for a team
- Configuring branch policies for compliance
- Setting up CI/CD pipelines with approvals
- Implementing security scanning in pipelines
- Troubleshooting deployment failures
- Configuring monitoring and alerts
- Managing secrets across environments
- Selecting deployment strategies for different scenarios

Important Concepts to Memorize

- **Branch policy options** and when to use each
 - **Deployment job syntax** vs regular job syntax
 - **Environment vs stage vs job** hierarchy
 - **Service connection types** and their use cases
 - **Agent capabilities** and demands
 - **Variable scopes:** Pipeline, stage, job levels
 - **Template types:** Extends, includes, parameters
-

Official Microsoft Learn Resources

Learning Paths

- [AZ-400: Get started on a DevOps transformation journey](#)
- [AZ-400: Development for enterprise DevOps](#)
- [AZ-400: Implement CI with Azure Pipelines and GitHub Actions](#)
- [AZ-400: Design and implement a release strategy](#)
- [AZ-400: Implement a secure continuous deployment](#)
- [AZ-400: Manage infrastructure as code](#)
- [AZ-400: Design and implement a dependency management strategy](#)
- [AZ-400: Implement continuous feedback](#)
- [AZ-400: Implement security and validate code bases](#)

Documentation

- [Azure DevOps Documentation](#)
- [Azure Pipelines YAML Reference](#)
- [GitHub Actions Documentation](#)
- [Azure Monitor Documentation](#)
- [Azure Key Vault Documentation](#)

Exam Information

- [AZ-400 Exam Page](#)
- [AZ-400 Study Guide](#)

Last Updated: December 2024 Exam Version: Based on skills measured as of November 2024